5 Componentes da arquitetura do SegCar

O **SegCar** é uma plataforma de telemetria veicular, rastreamento contínuo e segurança patrimonial em tempo real. O usuário acessa o sistema por meio de navegador web para acompanhar o deslocamento georreferenciado de veículos, visualizar odometria e alertas de perímetro, enquanto nós coletores embarcados (ou emuladores de telemetria via porta serial) transmitem dados de latitude, longitude, velocidade e satélites. As informações cadastrais, o histórico de trilhas e os índices analíticos residem em banco de dados relacional com tolerância a falhas.

A arquitetura reúne componentes e serviços bem definidos: interface cliente no navegador (Dashboard e Administração), servidor web e proxy reverso, camada de serviços de negócio em PHP 8, motor de regras espaciais (Haversine), agente despachante de telemetria serial/IoT e persistência híbrida em MySQL e arquivos de contingência. O desenho apresenta os componentes, suas integrações externas e a persistência local. [1–3]

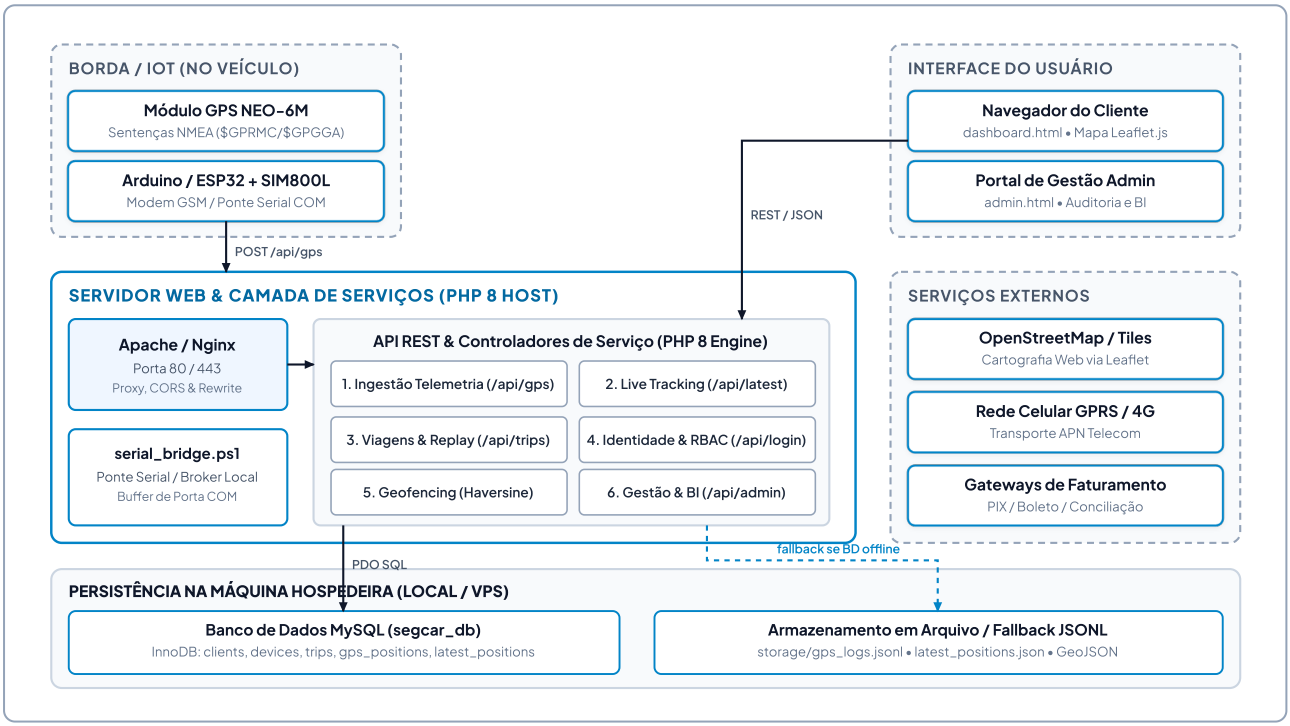


Figura 1 — Visão geral da arquitetura. Setas indicam solicitações ou fluxo de dados; respostas percorrem o caminho inverso. Linhas tracejadas indicam chaveamento de contingência/fallback. Cruzamentos sem ponto não representam conexão física. Fonte: elaboração a partir de [1–3].

O servidor Apache encaminha as requisições HTTP e serve os ativos estáticos aos clientes web. O script `serial_bridge.ps1` atua como despachante de telemetria local quando operando com placas conectadas via porta serial USB. A camada de serviços em PHP 8 realiza o controle de acesso, o processamento de regras espaciais e a gravação relacional no MySQL, dispondo de contingência em arquivos append-only (`gps_logs.jsonl`) caso o banco se torne indisponível.

5.1 Cliente e interface de acesso

O cliente é o navegador web utilizado pelos proprietários dos veículos e gestores de frota. Ele exibe as telas de autenticação (`login.html` e `registro.html`), o painel operacional de rastreamento (`dashboard.html`) e a central de controle gerencial (`admin.html`). Os scripts JavaScript utilizam a API Fetch de forma assíncrona para consultar o estado do veículo a cada intervalo configurável (1 a 5 segundos), renderizando dinamicamente a posição do marcador sobre os blocos cartográficos fornecidos pelo Leaflet.js.

O navegador não executa operações diretas de banco de dados nem possui acesso às chaves mestras de autenticação de hardware (`X-API-Key`). Todas as operações sensíveis, tais como encerramento de percursos, cálculo de métricas de faturamento e regras de geofencing, são centralizadas no backend para manter a integridade dos dados e impedir adulterações pelo cliente.

5.2 Frontend e proxy reverso

O servidor web (Apache 2.4 no ambiente XAMPP ou Nginx em implantações Linux) é responsável por servir diretamente os arquivos estáticos (HTML, CSS e JavaScript) e despachar as rotas de API para o interpretador PHP 8. O tráfego de entrada padrão utiliza a porta 80 (HTTP) ou 443 (HTTPS), compartilhando o mesmo domínio e porta para os recursos de interface e para os endpoints de serviço. [3]

Essa configuração simplifica a política de mesma origem (Same-Origin Policy), enquanto cabeçalhos `Access-Control-Allow-Origin` , `Access-Control-Allow-Methods` e `Access-Control-Allow-Headers` são manipulados para permitir integrações seguras com emuladores de GPS e pontes de hardware locais.

5.3 API e regras da plataforma

O núcleo do backend é composto por uma API HTTP estruturada em PHP 8 modular (`sketch_jul12a/api/public/index.php`). Ele gerencia a sessão de usuário, valida senhas com algoritmo criptográfico Bcrypt, processa os pacotes de telemetria recebidos dos nós coletores, alimenta as tabelas relacionais do MySQL e consolida estatísticas para visualização. [4]

Interface	Operação principal
`POST /api/gps`	Receber pacote de telemetria (lat, lng, velocidade, satélites, hora) autenticado por chave de hardware.
`GET /api/latest`	Recuperar a última coordenada válida e estado de movimento de um veículo em complexidade O(1).
`GET /api/history`	Consultar trilha histórica recente de pontos georreferenciados para traçado do percurso no mapa.
`GET /api/trips` e `/trips/positions`	Listar viagens consolidadas (origem, destino, km total) e recuperar coordenadas para replay visual.
`POST /api/login` e `/register`	Autenticar credenciais com verificação Bcrypt e efetuar cadastro de novos clientes e veículos.
`GET /api/clients` e `/client/update`	Consultar inventário de motoristas e sincronizar dados cadastrais de perfil e placa.
`GET /api/admin/devices`	Auditar módulos GPS, última conexão, indicador de status online/offline e chaves M2M vinculadas.
`GET /api/admin/metrics`	Consolidar KPIs em tempo real: total de clientes, carros monitorados, taxa de conectividade e receita (MRR).
`GET /health`	Verificar a disponibilidade do serviço HTTP e o status de conexão com o banco de dados relacional.

5.4 Fila de telemetria, buffering e ponte serial (Serial Bridge)

5.5 Worker e processamento de telemetria e viagens

O processamento contínuo de telemetria é executado pelo motor de viagens (*Trip Engine*) integrado à API. Ao receber cada ponto, o sistema avalia o deslocamento geodésico em relação à última posição conhecida. Viagens são identificadas e segmentadas autonomamente: o início é computado quando a velocidade supera o limiar de parada (5 km/h) e o veículo se afasta do perímetro seguro; o encerramento ocorre após 5 minutos consecutivos de repouso ou por comando de finalização. [1, 5]

Durante o percurso, o serviço acumula a distância total percorrida aplicando soma métrica de vetores, registra o tempo decorrido em segundos, identifica o pico de velocidade e calcula a velocidade média da rota. Ao mesmo tempo, atualiza a tabela `latest_positions` de forma atômica para permitir consultas imediatas pelo dashboard.

5.6 Motor de regras de telemetria e componentes do sistema

A inteligência analítica do SegCar distribui responsabilidades entre a borda embarcada no veículo e os algoritmos residentes no servidor. A tabela a seguir especifica os componentes técnicos, seu local de execução e sua função no ciclo de vida dos dados. [6]

Recurso	Execução	Responsabilidade
Módulo GPS NEO-6M	Borda (Hardware)	Recepção dos sinais de satélite, extração de sentenças NMEA e cálculo de latitude, longitude, altitude e HDOP.
Firmware C++ (Arduino)	Borda (MCU)	Amostragem temporizada (1 a 5s), empacotamento do payload em formato JSON e envio via porta UART/Serial.
Modem SIM800L / Ponte	Borda / Host	Transmissão dos pacotes via comandos AT/GPRS ou encaminhamento via script PowerShell com cabeçalho `X-API-Key`.
Geofencing (Haversine)	Backend (PHP)	Cálculo trigonométrico da distância entre a coordenada atual e a garagem para determinar entrada/saída de cerca.
Trip Engine & Métricas	Backend (PHP)	Segmentação automática de percursos, odometria geodésica acumulada e consolidação de velocidade média e máxima.
Autenticador RBAC & Bcrypt	Backend (PHP)	Validação segura de senhas via hash criptográfico de custo 12 e segregação de papéis (cliente vs administrador).
Circuit Breaker Local	Backend (PHP/FS)	Desvio automático da persistência para arquivo append-only (`gps_logs.jsonl`) na indisponibilidade do MySQL.

Diferente de sistemas que dependem exclusivamente de processamento em nuvens proprietárias, o SegCar executa sua camada algorítmica localmente, garantindo baixa latência na detecção de infrações de velocidade e violações de perímetro.

5.7 Serviços externos e integrações de terceiros

A camada de visualização cartográfica consome azulejos de mapas (*map tiles*) providos pelo consórcio aberto OpenStreetMap e estilizados via CartoDB através da biblioteca Leaflet.js. Esse modelo assegura que o mapa seja renderizado com alta performance diretamente na GPU do cliente, sem requisições onerosas ao servidor da aplicação. [7]

A conectividade de longa distância dos veículos utiliza a infraestrutura de redes móveis celulares (canais GPRS/4G das operadoras de telecomunicação através de chips M2M com APN privada). O módulo de faturamento permite a geração de cobranças por PIX e boletos bancários. Toda a arquitetura foi desenhada para ser independente de nuvem (*cloud-agnostic*), viabilizando implantações em VPS, servidores físicos locais com XAMPP ou contêineres Docker. [7, 8]

5.8 Catálogo e persistência de dados

O sistema SegCar implementa uma estratégia híbrida de armazenamento. A persistência relacional principal fica sob a custódia do MySQL (engine InnoDB), provendo garantias ACID e chaves estrangeiras. Para otimizar as consultas contínuas de alta frequência realizadas pelos mapas, mantém-se uma tabela desnormalizada de acesso rápido (`latest_positions`) indexada por `device_id`, evitando varreduras completas no histórico massivo de telemetria. Em paralelo, arquivos locais funcionam como buffer de contingência. [5, 9]

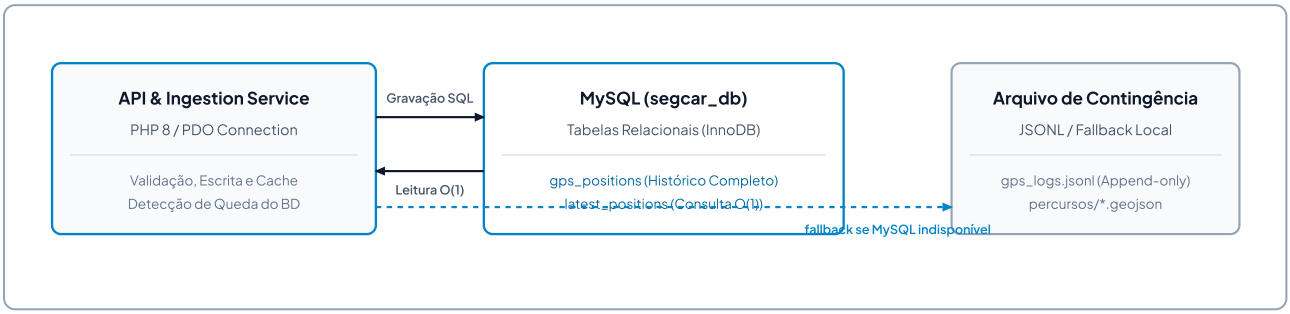


Figura 2 — Relação entre banco relacional, cache de última posição e fallback em arquivo. A sincronização ocorre a cada pacote recebido, com chaveamento instantâneo para armazenamento em arquivo caso ocorra interrupção de conectividade com o banco. Fonte: [5, 7–9].

Local	Conteúdo	Uso
`segcar_db.gps_positions`	Latitude, longitude, velocidade, altitude, satélites, HDOP e timestamp UTC.	Histórico contínuo e ordenado para traçado de rotas e auditoria de viagens.
`segcar_db.latest_positions`	Última coordenada válida, velocidade, status e horário recebido por veículo.	Tabela de acesso instantâneo em complexidade O(1) para polling de alta frequência pelo mapa.
`segcar_db.trips`	Percursos fechados: origem, destino, km total, tempo e velocidade média.	Relatórios operacionais e recuperação de viagens históricas para replay.
`segcar_db.clients / devices`	Cadastros, hashes Bcrypt, vínculos de hardware e chaves de autenticação.	Controle de acesso (RBAC), autorização de endpoints e gestão do inventário de frota.
`segcar_db.plans / subscriptions`	Planos (Básico, Frota Pro), faturas, pagamentos e datas de vencimento.	Módulo financeiro e controle do ciclo de vida das assinaturas de rastreamento.
`api/storage/gps_logs.jsonl`	Linhas JSON append-only com o payload bruto dos pacotes de telemetria.	Armazenamento de contingência local que garante zero perda se o banco estiver fora.
`api/storage/latest_positions.json`	Dicionário JSON contendo a última localização de cada `device_id`.	Cache em disco para continuidade das consultas do mapa durante manutenções no MySQL.
`percursos/*.geojson`	Coleção de pontos e linhas no padrão geoespacial RFC 7946 GeoJSON.	Exportação e interoperabilidade com softwares GIS (QGIS, ArcGIS e Google Earth).

Cada registro de telemetria na tabela `gps_positions` possui chave estrangeira indexada para a tabela `trips` com regra `ON DELETE SET NULL`, garantindo a preservação dos pontos geográficos mesmo na hipótese de exclusão do cabeçalho da viagem. A integridade cadastral de clientes e dispositivos utiliza restrição `ON DELETE CASCADE`. [5, 9]

5.9 Fluxo de autenticação

O acesso seguro de usuários à plataforma SegCar é baseado em credenciais cifradas e segregação de papéis (RBAC). O fluxo é executado pela rota `POST /api/login` dentro do backend PHP, recebendo o e-mail ou CPF e a senha informada pelo operador. As requisições passam pela camada de proteção do servidor web Apache. [7]

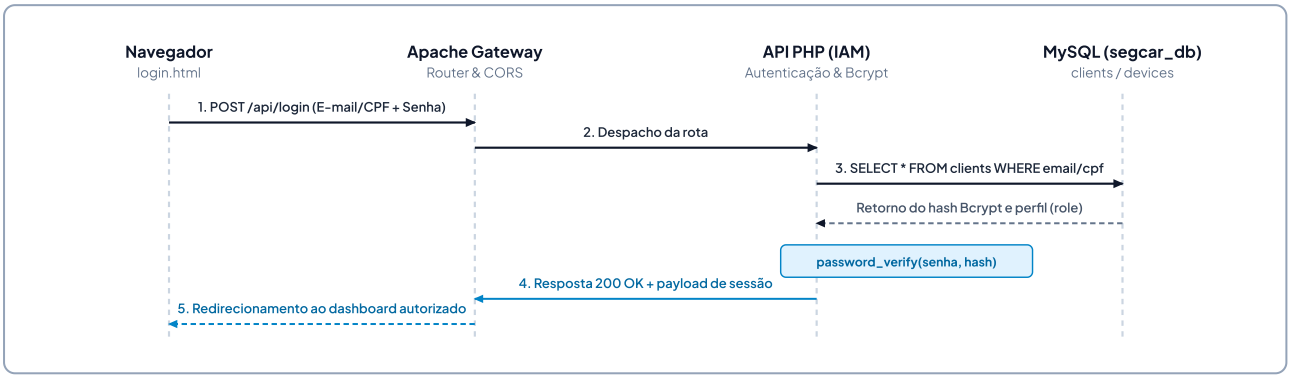


Figura 3 — Sequência de login e verificação de privilégios. As chamadas de verificação criptográfica e consulta de perfil foram detalhadas para demonstrar o controle de permissão baseado em papéis (RBAC). Fonte: [7, 8].

1. **1. Início do acesso.** O usuário submete o formulário com login e senha. O script JavaScript monta o envelope JSON e despacha a requisição HTTP POST para o endpoint `/api/login` via fetch assíncrono.
2. **2. Localização da conta.** A API realiza consulta parametrizada via PDO com prepared statement na tabela `clients`, buscando por e-mail ou CPF, prevenindo qualquer possibilidade de injeção SQL.
3. **3. Validação do hash criptográfico.** A senha em texto plano informada na requisição é comparada contra o hash armazenado utilizando a função nativa `password_verify()` com algoritmo Bcrypt (RFC 7693). Senhas nunca repousam em texto claro no banco de dados.
4. **4. Resolução de ativos e permissões.** Confirmada a autenticidade, o sistema consulta os veículos associados ao usuário na tabela `devices` e constrói o payload de sessão contendo `role` (cliente ou admin), `device_id` principal e dados cadastrais.
5. **5. Entrada na plataforma.** A API responde com status HTTP 200 OK. O frontend armazena a sessão em `sessionStorage` e realiza o redirecionamento: usuários com perfil `client` são conduzidos ao mapa operacional (`dashboard.html`), enquanto o perfil `admin` desbloqueia o centro gerencial de frotas e auditoria (`admin.html`).

Caso o usuário informe credenciais inexistentes ou incorretas, a API retorna código 401 Unauthorized com payload padronizado, impedindo a enumeração de usuários e bloqueando a continuidade do acesso no navegador.

5.10 Fluxo de envio e classificação

A recepção dos dados brutos de GPS e a sua consolidação analítica são etapas distintas. A API recebe o pacote via HTTP POST, valida a autenticidade da chave M2M do transmissor e delega a gravação tanto para a base relacional quanto para o cache rápido de mapa. [4–6]

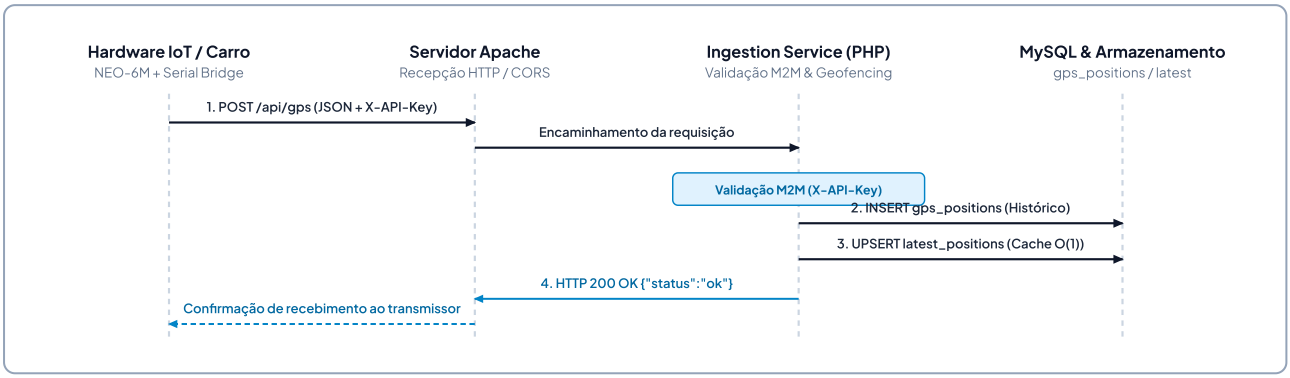


Figura 4 — Processamento de um pacote de telemetria após seu recebimento pela API. A persistência relacional e a atualização do cache rápido ocorrem de forma paralela no mesmo ciclo de requisição. Fonte: [4–6, 8].

1. **1. Recebimento e autenticação M2M.** A API intercepta o pacote HTTP POST no endpoint `/api/gps` e extrai o cabeçalho `X-API-Key`. A chave é checada contra a tabela `devices` para certificar que o identificador do veículo (`device_id`) está autorizado a transmitir.
2. **2. Sanitização e cálculo espacial.** Os campos numéricos de latitude, longitude, velocidade e altitude são normalizados. O motor de geofencing calcula a distância geodésica em relação ao ponto base do veículo, determinando se a posição atual configura perímetro seguro ou deslocamento ativo.
3. **3. Persistência dupla otimizada.** O ponto é inserido no histórico contínuo (`gps_positions`) e simultaneamente registrado na tabela de acesso rápido (`latest_positions`) por meio de comando `INSERT ... ON DUPLICATE KEY UPDATE`. Caso o banco de dados falhe, o manipulador de erro aciona o circuit breaker, salvando a linha em `gps_logs.jsonl`.
4. **4. Acompanhamento em tempo real.** A API devolve o código HTTP 200 OK com payload de confirmação. O dashboard web, em seu ciclo regular de polling (`GET /api/latest`), lê o registro recém-gravado na tabela de cache e anima o marcador no mapa.

Se o envio da telemetria ocorrer sem sinal de satélites adequado (campo `satelites = 0` ou `hdop > 5.0`), a API aceita o pacote para fins de auditoria de conexão, porém sinaliza a flag de baixa precisão para evitar saltos visuais errôneos no mapa do usuário. [5]

5.11 Fluxo de busca e análise espacial (Geofencing)

A plataforma executa a avaliação espacial contínua das coordenadas transmitidas, comparando a posição do veículo contra pontos de ancoragem pré-definidos (como a garagem ou sede da empresa). A tomada de decisão de perímetro é calculada localmente no backend. [4, 6, 10]

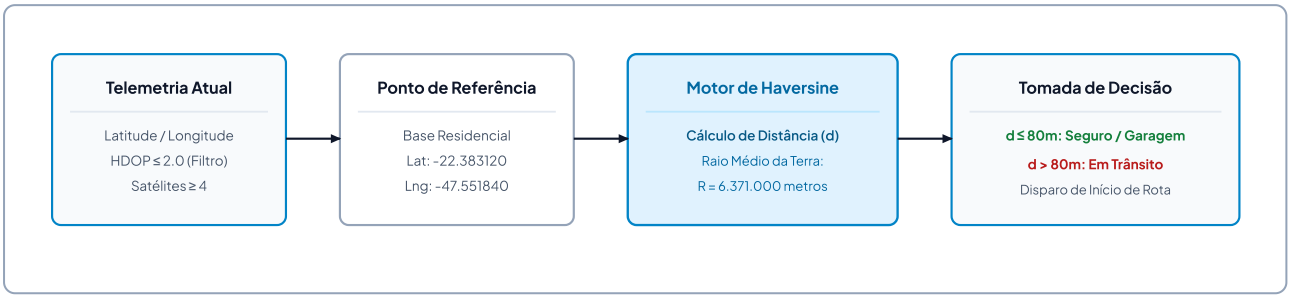


Figura 5 — Pipeline de cálculo espacial e decisão de cerca virtual. A normalização das coordenadas em radianos e a aplicação trigonométrica ocorrem antes da gravação do status do veículo. Fonte: [4, 6, 10].

A distância esférica ortodrômica entre as coordenadas é formulada através da relação de Haversine:

$$d = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta \varphi}{2}\right) + \cos(\varphi_1)\cos(\varphi_2)\sin^2\left(\frac{\Delta \lambda}{2}\right)}\right)$$

Onde $R = 6.371\text{ km}$ representa o raio médio da Terra, φ_1, φ_2 correspondem às latitudes de origem e destino expressas em radianos, e $\Delta \varphi, \Delta \lambda$ representam as variações angulares diferenciais de latitude e longitude.

Filtragem de ruído de satélite: Ambientes urbanos e garagens subterrâneas produzem dispersão de sinal (*jitter*). O sistema descarta amostras com diluição horizontal de precisão elevada ($\text{HDOP} > 2.0$) ou com constelação reduzida ($\text{satelites} < 4$), impedindo falsos disparos de violação de cerca enquanto o veículo permanece imóvel.

Cenário prático demonstrativo: No percurso de testes mapeado pelas vias de Rio Claro-SP (partindo do Jardim Floridiana em direção ao Centro), o veículo é mantido no estado seguro enquanto sua distância linear for inferior a 80 metros da garagem residencial. No instante em que cruza a Avenida Visconde de Rio Claro e o raio atinge 81 metros, o backend atualiza o status para *"Em Trânsito"*, abre um novo ciclo de viagem na tabela `trips` e atualiza a interface do motorista em tempo real.

5.12 Dependências e indisponibilidade

A distribuição funcional dos componentes permite isolar com precisão os impactos de falhas parciais de infraestrutura. A matriz a seguir detalha o comportamento preventivo implementado no código-fonte para tolerância a falhas operacionais. [1, 4–8]

Componente indisponível	Efeito principal	Continuidade ou tratamento
Servidor Web (Apache)	Interfaces web e endpoints REST tornam-se inacessíveis para navegadores e nós coletores.	Módulos de telemetria retêm pacotes em buffer de memória ou EEPROM local, efetuando novas tentativas com recuo exponencial (*retry*).
API PHP	Falha no tratamento de requisições dinâmicas e retornos de erro 500 ou timeout.	Arquivos estáticos já carregados no cliente exibem indicador de desconexão; a ponte serial continua armazenando dados da porta COM.
Banco de Dados MySQL	Indisponibilidade nas operações de INSERT e SELECT nas tabelas relacionais do sistema.	O mecanismo de contingência desvia a telemetria para o arquivo `storage/gps_logs.jsonl`; consultas do mapa utilizam `latest_positions.json`. Zero perda de dados.
Módulo GPS (NEO-6M)	Perda de sinal dos satélites (ex.: em túneis ou garagens cobertas).	O firmware envia pacotes com a flag de satélites zerada ou mantém a última coordenada conhecida até a recuperação do sinal (*cold start*).
Rede Celular (GPRS/4G)	Impossibilidade de transmissão remota dos pacotes entre o carro e a internet.	O modem SIM800L armazena o histórico em fila interna de comandos; no laboratório, a ponte serial USB garante o tráfego contínuo.
Provedor de Tiles (OSM)	Falha no carregamento dos blocos cartográficos no mapa do navegador.	A interface continua funcionando normalmente: velocímetro, coordenadas e indicadores operacionais permanecem ativos sobre fundo cinza.

Limites da configuração atual

A arquitetura implementada encontra-se consolidada em uma única máquina hospedeira (instância unificada via XAMPP no Windows ou VPS Linux). Embora os módulos de software sigam os princípios de separação de responsabilidades de SOA, a concorrência de I/O em disco e de conexões simultâneas ao MySQL compartilha os mesmos recursos de CPU e memória do sistema operacional.

O mecanismo de contingência em arquivos append-only (`gps_logs.jsonl`) atende perfeitamente a quedas temporárias de banco de dados para dezenas de veículos, porém não implementa concorrência distribuída de alta vazão com garantia de ordenação estrita em caso de expansão para milhares de dispositivos simultâneos.

Em implantações produtivas de grande porte, recomenda-se a segregação física dos nós: isolando a ingestão HTTP por meio de um broker de mensageria assíncrona dedicado (como Redis ou RabbitMQ), adotando cluster de banco de dados com réplica de leitura e executando o servidor web sob terminação SSL/TLS com balanceamento de carga. [1]

5.13 Organização arquitetural

O SegCar combina uma arquitetura cliente-servidor rica, comunicação orientada a serviços e integração assíncrona de telemetria. As responsabilidades de apresentação visual, controle de identidade, ingestão de sinais de hardware, cálculo espacial e inteligência financeira estão rigorosamente compartimentadas em controladores de domínio independentes.

Essa organização concretiza os princípios de **Arquitetura Orientada a Serviços (SOA)**: cada serviço expõe uma capacidade de negócio clara, interage exclusivamente por meio de contratos padronizados (JSON sobre HTTP/REST) e mantém baixo acoplamento (*loose coupling*). Embora executados sobre um mesmo monólito modular no ambiente de hospedagem unificado, a independência de contratos garante que qualquer serviço possa ser refatorado para microsserviços autônomos sem impactar as interfaces consumidoras.

A decomposição em componentes e serviços segue o modelo formal de exposição adotado na referência da UFRJ. O sistema SegCar não implementa CORBA: não utiliza ORB (*Object Request Broker*), IDL (*Interface Definition Language*), nem rotinas de *stubs* e *skeletons* binários legados. Em seu lugar, adota o padrão contemporâneo da Web: transporte HTTP/1.1 seguro, serialização em JSON, chaves M2M em cabeçalhos HTTP e persistência relacional com cache indexado em complexidade O(1). [1–3, 11]

5.14 Referências e rastreabilidade

Versão do projeto consultada em 21 de outubro de 2026, release v2.0 (Produção Local / XAMPP). Os arquivos e módulos referenciados apontam para a estrutura do diretório `c:/xampp/htdocs/segcar`.

[1] Apache HTTP Server (XAMPP v3.3.0) — Servidor web, roteamento e suporte a PHP 8.5.
[2] Documentação Técnica da Arquitetura SegCar — Especificação de Serviços e Telemetria.
[3] Nginx / Apache — Páginas estáticas, proxy reverso e encaminhamento de rotas HTTP.
[4] API PHP Modular — Controladores RESTful, contratos JSON e tratamento de telemetria.
[5] Schema Relacional MySQL — Modelo de dados InnoDB, índices e integridade referencial (`schema.sql`).
[6] Geofencing & Fórmula de Haversine — Algoritmo de cálculo geodésico de distâncias e cercas virtuais.
[7] Segurança Criptográfica Bcrypt (RFC 7693) — Autenticação e gestão segura de senhas.
[8] Hardware IoT Embarcado — Módulo GPS GY-NEO-6M, microcontrolador e firmware GSM (`sketch_jul12a.ino`).
[9] Serial Bridge & Buffer de Mensageria — Script PowerShell de enlace serial-HTTP (`serial_bridge.ps1`).
[10] Especificação GeoJSON (RFC 7946) — Formato aberto para dados espaciais e trilhas de percurso.
[11] UFRJ — Componentes da Arquitetura de Software e Sistemas Distribuídos.

Detalhamento dos módulos de código: Os scripts `sketch_jul12a/api/public/index.php`, `sketch_jul12a/api/database/schema.sql`, `sketch_jul12a/sketch_jul12a.ino` e `serial_bridge.ps1` implementam, respectivamente, o roteamento de serviços REST, a estrutura de persistência InnoDB com índices analíticos, o firmware de leitura satelital NMEA e o despachante de mensageria serial. As interfaces de visualização em `dashboard.js` e `admin.js` consom os contratos padronizados de forma desacoplada.

As figuras e diagramas foram elaborados especificamente para este documento com base nos fluxos e componentes reais do repositório SegCar. A página da UFRJ é a referência metodológica de organização por componentes e discussão arquitetural, sem equivalência tecnológica com CORBA.